Unit - I

Algorithm:- An algorithm is a sequence of definite and effective instructions, which terminates with the production of correct output from the given input.

Characteristics of Good Algorithm :-

1. Input
2. Output
3. Definitness
4. Effectiveness
5. Terminates
6. Timelessness

Q. What is analysis ?

Ans:- Analysis of algorithm is a field in Computer Science whose overall goal is an understanding of the Complexity of algorithm (in terms of time and space) also known as execution time and storage (space) requirement taken by that algorithm.

In short, finding efficiency of algorithm is Analysis.

Complexity:-

Complexity refers to the rate at which the required storage or consumed time grows as a function of the problem size. The absolute growth depends on the machine used to execute the program, the compiler used to compile and many other factor.

In general the term Complexity given any where simply refers to the running time of the algorithm.

There are 3 cases in general to find the complexity function $f(n)$.

1) Best Case :- The minimum value of $f(n)$ for any possible input.

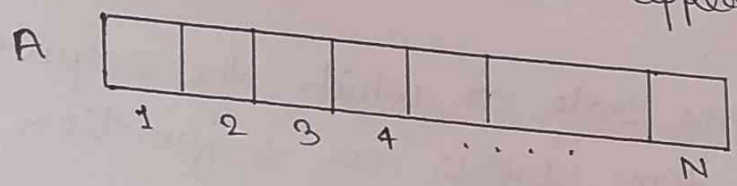
2) Worst Case :- The maximum value of $f(n)$ for any possible input.

3) Average Case :- The value of $f(n)$ which is in between the maximum and minimum for any possible input.

Generally the average case implies the expected value of $f(n)$.

To understand the best, worst and average case of an algorithm consider a linear array $A[1, 2, \cdots n]$ where the array $A[n]$ contain n elements.

Suppose you want either to find the location (LOC) of a given element (Say x) in the given array $A[n]$ or display a message such as LOC = 0, to indicate that x does not appear in $A[n]$.



Algorithm (Linear Search)

/* INPUT : A linear list A with n element and Search element x.

OUTPUT : Find the LOC of x in the array (by returning an Index) or return LOC = 0 to indicate x is in not present in A[N].

1. [initialize]
2. Set K=1 and loc=0
2. Repeat step 3, 4 and 5 while (LOC==0 && K<=n)
3. if (x==A[K])
   then
4. LOC=K
5. K=K+1
6. if (LOC==0)
7. printf("x is not present in given Array");
8. else
9. print ("x is present at location K");
10. Exit (Terminate).

Now analysing the above algorithm

The complexity of LSA is given by the number C of comparision between x and array element A[K].

1) Best Case :- Clearly the best case occurs when x is the first element in array i.e x = A[LOC].
In this case C(n) = 1

x = A[LOC]
C(n) = 1

2) Worst Case :- The worst case occurs when x is the last element in the array A[n] or x is not present in the given array.

i.e C(n) = n

3) Average Case :- We assume that Search element x appears in array A[n] and it is equally likely to occur at any position in array. Here the no. of comparision (C) can be any number (1, 2, 3, ... n) and each number

occurs with the probability $P = \frac{1}{n}$, then

$$C(n) = 1 \cdot \frac{1}{n} + 2 \cdot \frac{1}{n} + \cdots n \cdot \frac{1}{n}$$

$$= \frac{1}{n}(1 + 2 + 3 + \cdots + n)$$

$$= \frac{1}{n} \times \frac{n(n+1)}{2}$$

$$C(n) = \frac{n+1}{2}$$

It means the average no. of comparision needed to find the location of x is approximately equal to half of the numbers in array.

Asymptotes :—

There are three basic asymptotic (i.e $n \to \infty$) notation which are used to express the running time of an algorithm in terms of function whose domain is the set of Natural number ($N = 1, 2, 3, \cdots$)

There are :—

1. O (Big Oh) :— This Notation is used to express upper bound (max^m steps / Worst Case) required to solve the problem.
2. Ω (Big 'Omega') :— It is used to express lower bound (Min^m steps / Best Case) required to solve a problem.
3. Θ (Big 'Theta') :— It is used to express both upper and lower bound (average case on a function).

Asymptotic notations gives the rate of growth i.e performance of the run time for sufficiently large input sight i.e $n \to \infty$ and is not a major of the particular run time for specific input size.

There are basically 5 fundamental techniques which are used to design algorithm efficiently.

① Divide and Conquire:-
- Binary Search
- Multiplication of 2 n-bit numbers
- Quick Sort
- Heap Sort

② Greedy Method :-
- KnapSack Problem
- Minimum Cost Search tree
- Kruskal's algorithm
- Prim's algorithm
- Single source Sortest path problem
- Dijakstra Algorithm

③ Back Cracking

④ Dynamic Programming

⑤ Branch and Bound.

Generally the time complexity of an algorithm is given by the number of steps taken by the algorithm to complete the function it was written for.

By How to Calculate the time complexity of any program ?

The time required by ~~the~~ an algorithm is determined by the number of elementry operations.

The following primitive operations that are independent from the programming language are used to calculate the running time :—

① Assigning a value to the variable.
② Calling a function.
③ Performing an arithmetic operation.
④ Comparing two variables.
⑤ Indexing into an array or following a pointer reference.

```
int Sum(int n)
 {
   int i, Sum;
1.   Sum = 0;                          = 1
2.   for (i=0; i<n; i++)               = n+1
      {
3.      Sum = Sum + i*i;               = n
      }
4.   return Sum;                       = 1
 }
```

Total time = 1 + (n+1) + n + 1
= 2n + 3

O(n) = n

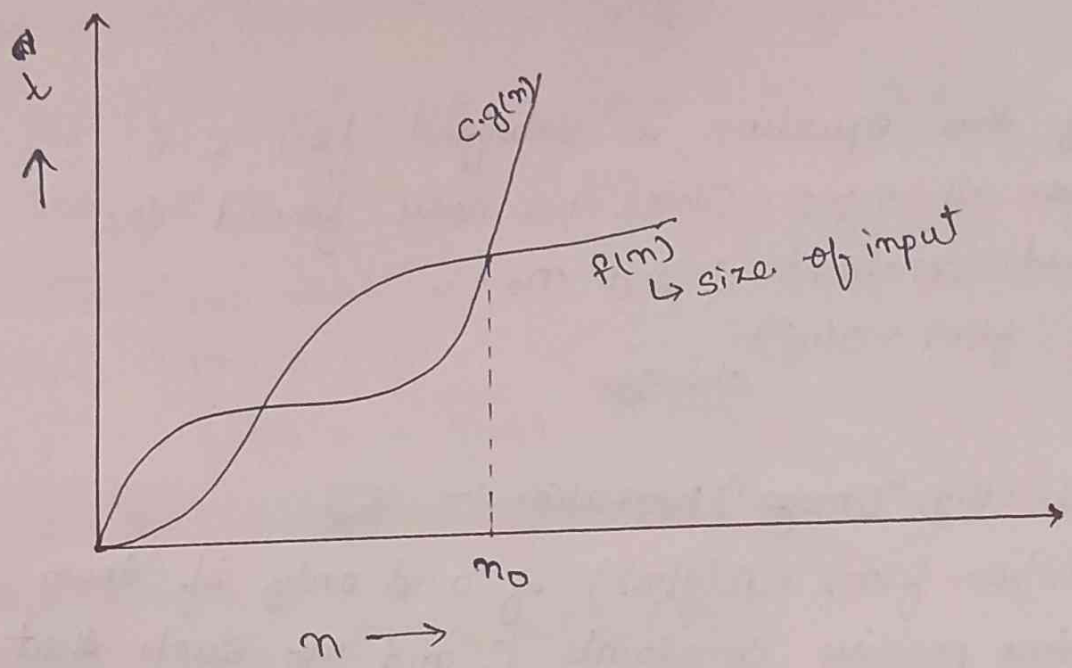
To determine the running time of this program we have to count the number of statement that are executed in this procedure.

① The O(Big, oh) Notation :- O(Big, 'oh') Notation is used to express any asymptotic upper bound for any given function f(n).

Let $f(n)$ and $g(n)$ are two positive function. is from the set of natural number to the positive real number.

We say that the function $f(n) = O(g(n))$, if there exists two positive constant C and n_0 such that $f(n) \leq C \cdot g(n)$ $\forall$ $n \geq n_0$, $C > 0$, $n \geq 1$.

$$f(n) \leq C \cdot g(n) \; ; \; \forall \; n \geq n_0$$



$f(n) = O(g(n))$

$f(n) = 3n+2$

$3n+2 \leq C \cdot g(n)$

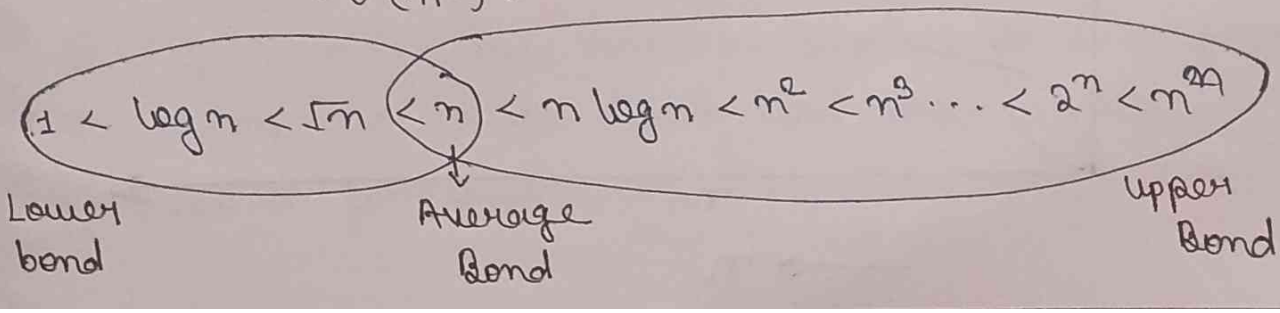
$3n+2 \leq 3n+2n$

$3n+2 \leq 5n$

$f(n) = O(n)$ ✓

$= O(n^2)$ ✓

$= O(n^3)$ ✓



Ex: For the function defined by $f(n) = 2n^2 + 3n + 1$ show $f(n) = O(n^2)$.

Sol: To show $f(n) = O(n^2)$ we have to show that function should satisfy $f(n) \leq c \cdot g(n)$ $\forall$ $n \geq n_0$.

$$f(n) \leq c \cdot g(n)$$

$$2n^2 + 3n + 1 \leq 2n^2 + 3n^2 + n^2$$

$$2n^2 + 3n + 1 \leq 6n^2$$

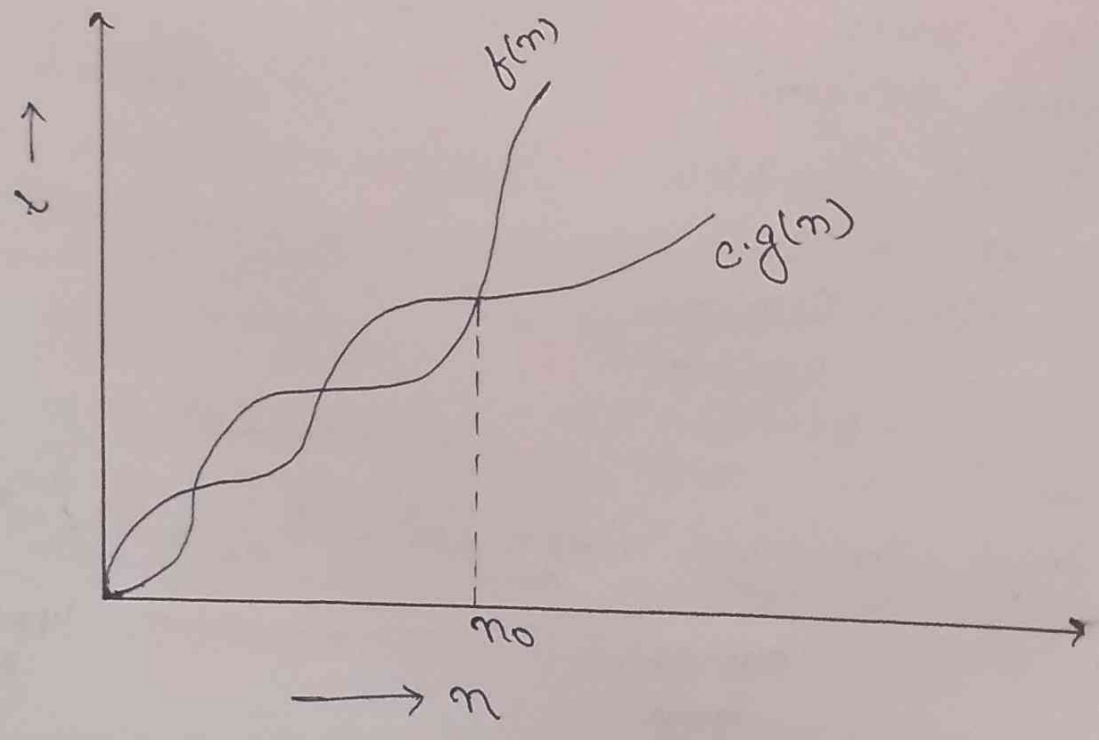
c → 6, $g(n)$ → n^2

Clearly this equation is satisfied for $c = 6$ and for all $n \geq 1$. Since we have found the required constant c and n_0.

Hence, $f(n) = O(n^2)$ showed

(2) The Ω (Big, "Omega") Notation :-

The function $f(n) = \Omega(g(n))$ if and only if there exist two positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ $\forall$ $n \geq n_0$.



Hence all values of $f(n)$ always lie on n_0 or above $g(n)$.

$$f(n) \geq c \cdot g(n)$$

$$3n+2 \geq c \cdot n$$

$3n+2 \geq 5n$; True for $n=1$

$$f(n) = \Omega(n)$$

For $n = \sqrt{n}$, $\log n$, 1

True upper tight bound i.e n

Q. If $f(n) = 2n^3 + 3n^2 + 1$ and $g(n) = 2n^2 + 3$ Show that $f(n) = \Theta(g(n))$.

Sol To show that $f(n) = O(g(n))$ we have to show $f(n) \geq c \cdot g(n)$ $\forall$ $n \geq n_0$

$$f(n) \geq c \cdot g(n)$$

$$2n^3 + 3n^2 + 1 \geq c \cdot (2n^2 + 3)$$

Put $c=1$, $n=1$

$$2(1)^3 + 3(1)^2 + 1 \geq c \cdot (2(1)^2 + 3)$$

$$6 \geq 5 \quad \text{True}$$

Clearly this equation satisfy for $c=1$ and for all $n \geq 1$.

Since, we have found the required constant c and $n_0 = 1$, Hence $f(n) = O(g(n))$

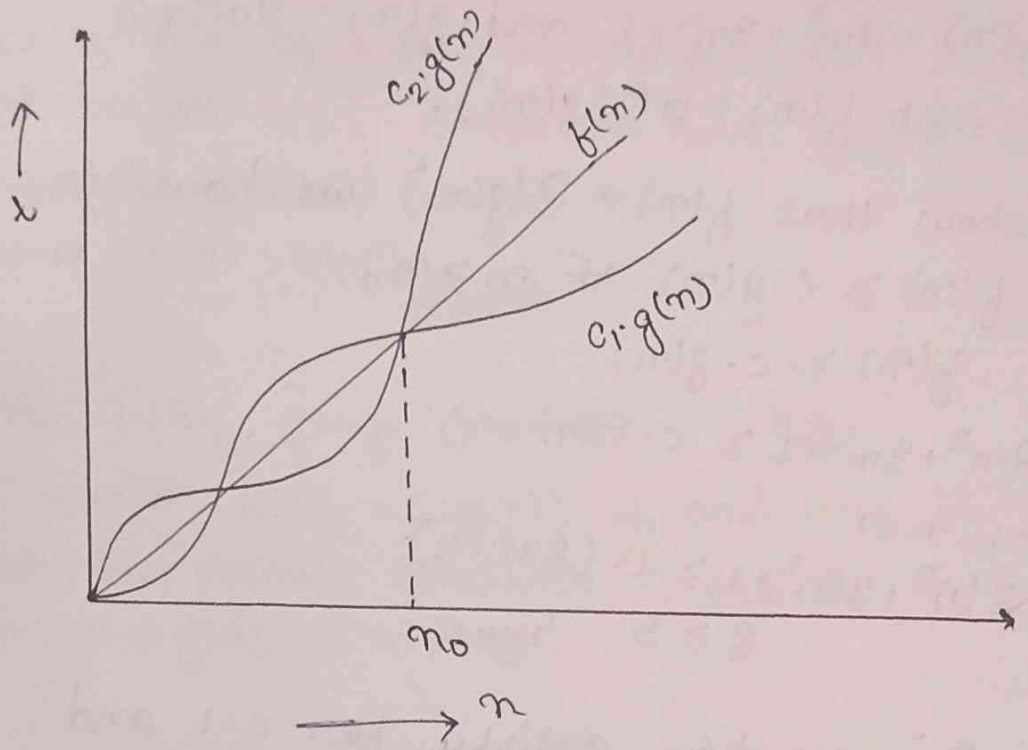
Proved//

(3) Θ (Big. "Theta"):- This notation provide simultaneously both asymptotic lower bond and asymptotic upper bond for given function.

Let $f(n)$ and $g(n)$ are two +ve function each from the set of natural number to the +ve real number.

we say that the function $f(n) = \Theta(g(n))$ if and only if there exists three +ve constant c_1, c_2 and n_0 such that

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \forall\ n \geq n_0$$



$f(n) = 3n+2$

$c_1 \cdot g(n) \leq 3n+2 \leq c_2 \cdot g(n)$

Case -(1)

$c_1 \cdot g(n) \leq 3n+2$

~~Put $c_1 = 5$, $n \to$~~

$5n \leq 3n+2$

$\therefore c_1 = 5$

Case -(2)

$3n+2 \leq c_2 \cdot g(n)$

$3n+2 \leq 5n$

$C_2 = 5$

Q. The function $f(n) = 2n^3 + 3n^2 + 1$ and $g(n) = 2n^3 + 1$
Show that i) $f(n) = \Theta(n)$
ii) $f(n) \neq \Theta(n^2)$

Q. Define an algorithm? What are the various properties of good algorithm.

Q. What are the various fundamental techniques to design an algorithm. Also write two problem for each that follow these techniques.

Q. What is asymptotic Notation? Differentiate between asymptotic notation Big. Oh, Big. Omega and Big. Theta.

Q. Define time complexity

Q. Let $f(n)$ and $g(n)$ be two asymptotic tve function. Prove or disprove following using the basic definition of Big. O, Big. Omega and Big. Theta.

(1) $4n^2 + 7n + 12 = O(n^2)$

(2) $\log n + \log(\log n) = \Omega(\log n)$

(3) $3n^2 + 7n - 5 = \Theta(n^2)$

(4) $2^{(n+1)} = \Omega(2^n)$

(5) $33n^3 + 4n^2 = \Omega(n^4)$

Iterative function:-

```
A(n)
{
  int i, j;
  for(i=1; j<=n; j++)
  {
    for(j=1; j<=n; j++)
    {
      printf("BY");
    }
  }
}
```

$O(n^2)$

```
A(n)
{
  int i=1, S=1;
  while(S<=n)
  {
    i++;
    S = S+i;
    printf("A.K");
  }
}
```

S	1	3	6	10	15	.....	> n
i	1	2	3	4	5	.....	Kth

Let me say that value of S will be $1+2+3+\cdots+K$

$= \frac{K(K+1)}{2}$

$\therefore \frac{K(K+1)}{2} > n$

$K^2 > n$

$K = O(\sqrt{n})$

Q4
```
A(n)
{
  int i,j,k;
  for(i=1; j<=n; j++)
  {
    for(j=1; j<=i; j++)
    {
      for(k=1; k<=100; k++)
      {
        printf("AK");
      }
    }
  }
}
```

i=1 time	i=2	i=3	... i=n time
j=1 time	j=2 time	j=3 time	... j=n time
K=100 time	K=2×100 time	K=3×100 time	... K=n×100 time

Total time $= 100 + 2\times100 + 3\times100 + \cdots + n\times100$

$= 100(1+2+3+\cdots n)$

$= 100\,\frac{n(n+1)}{2}$

$= 100\,\frac{n^2+n}{2}$

$= O(n^2)$

Q5
```
A(n)
{
  int i,j,k;
  for(i=1; i<=n; j++)
  {
    for(j=1; j<=i^2; j++)
    {
```

```
for(k=1; k<=n/2; k++)
  {
    printf("AK");
  }
  }
}
```

$i=1$ time	$i=2$ times	$i=3$ times	...	$i=n$ times
$j=1$ time	$j=4$ times	$j=9$ times	...	$j=n^2$ times
$k=\frac{n}{2}$ times	$k=4\times\frac{n}{2}$ times	$k=9\times\frac{n}{2}$ times	...	$k=n^2\times\frac{n}{2}$ times

Total time $= \frac{n}{2} + 2^2\frac{n}{2} + 3^2\frac{n}{2} + \cdots n^2\frac{n}{2}$

$= \frac{n}{2}(1^2 + 2^2 + 3^2 + \cdots n^2)$

$= \frac{n}{2}\frac{n(n+1)(2n+1)}{6}$

$= O(n^4)$

```
A(n)
{
  int i;
  for(i=1; i<=n; i=i*2)
    {
      printf("AK");
    }
}
```

$i=$	1	2	4	8	16	....	$\leq n$
	2^0	2^1	2^2	2^3	2^4	...	2^k

$2^k \leq n$

$\therefore k = O(\log n)$

Q) A(n)

```
{
 int i,j,k;
 for(i= n/2 ;i<=n;i++)          — n/2
 {
  for(j=1;j<=n/2;j++)           — n/2
  {
   for(k=1;k<=n;k=k*2)          — log2n
   {
    printf("AK");
   }
  }
 }
```

Total time = $\frac{n}{2} \times \frac{n}{2} \times \log_2 n$

$= O(n^2 \log n)$

Q) A{n}

```
{
 int i,j,k;
 for(i=n/2 ;i<=n;i++)           — n/2
 {
  for(j=1;j≤n;j=2*j)            — log2n
  {
   for(k=1;k<=n;k=k*2)          — log2n
   {
    printf("AK");
   }
  }
 }
```

Total Time = $\frac{n}{2} \times \frac{n}{2} \times \log_2 n \times \log_2 n$

$= O(n(\log_2 n)^2)$

Q) A(n)

```
{
 for(i=1;i<=n;i++)
 {
  for(j=1;j<=n;j=j+i)
  {
   printf("AK");
  }
 }
```

Recursive Algorithm :- A recursive algorithm is one which makes a recursive call to itself with smaller input.

A reoccurance relation is an equation or inequality that describe a function in terms of its value on smaller inputs or as a function of preceding (or lower) terms.

Like all recursive function, a reoccurance also consist of two steps.

(1) Basic Steps:- Basic step have one or more constant values which are used to terminate reoccurance. It is also known as initial condition or base condition.

(2) Recursive Step:- Recursive step is used to find new terms from the existing term. This formula is called reoccurance relation or recursive formula.

Method for solving recurrence relation :-

1. Back Substitution Method / Substitution
2. Iteration method
3. Recursive Tree Method
4. Master Method

1. Back Substitution Method / Substitution

```
A(n)
{
if (n>1)                    — 1
  return (A(n-1));          — T(n-1)
}
```

Total time required for n inputs

$$T(n) = 1 + T(n-1) \quad \text{— (1)}$$

By Substitution Method

$$T(n-1) = 1 + T(n-2) \quad \text{— (2)}$$

Again

$$T(n-2) = 1 + T(n-3) \quad \text{— (3)}$$

$$\vdots$$

$$T(1) = 1$$

$$\therefore T(n) = 1 + T(n-1) \quad \text{if } n > 1$$

$$T(n) = 1 \quad \text{if } n = 1 \text{ (Base Condition)}$$

Substitute eqn — (1) from eqn — (2)

$$T(n) = 1 + 1 + T(n-2)$$

$$T(n) = 2 + T(n-2) \quad \text{— (4)}$$

Put T(n-2) in eqn — (4)

$$T(n) = 2 + 1 + T(n-3)$$

$T(n) = 3 + T(n-3)$

$\vdots$

$T(n) = k + T(n-k)$ ..... In general equation

For Base condition;

$n - k = 1$

$k = n-1$

Put $k = n-1$ in General equation

$T(n) = n-1 + T(n-n+1)$

$T(n) = n-1 + T(1)$

$= n-1+1$ (As $T(n) = 1$ if $n = 1$)

$T(n) = n$

∴ Time complexity = $O(n)$

Q) $T(n) = n + T(n-1)$; $n > 1$ — (1)

$= 1$; $n = 1$

By back substitution method

$T(n-1) = n-1 + T(n-2)$ — (2)

Again

$T(n-2) = n-2 + T(n-3)$ — (3)

Again

$T(n-3) = n-3 + T(n-4)$ — (4)

Substituting eqn — (1) from (2)

$T(n) = n + n-1 + T(n-2)$

$T(n) = 2n-1 + T(n-2)$ — (5)

Put the value of $T(n-2)$ in eqn — (5)

$T(n) = 2n-1 + n-2 + T(n-3)$

$T(n) = 3n-3 + T(n-3)$

$\vdots$

$T(n) = kn - k + T(n-k)$ ... In general equation

For base condition:

$n-k = 1$

$k = n-1$

Put $k = n-1$ in general equation

$T(n) = (n-1)n - (n-1) + T(n-n+1)$

$= n^2 - n - n + 1 + T(1)$

$= n^2 - 2n + 1 + 1$ (as $T(1) = 1$)

$T(n) = n^2 - 2n + 2$

∴ Time Complexity $= O(n^2)$

(3) Recursion Tree Method :-

A recursion tree is a convenient way to visualize what happens when a recurrance is iterated. It is a pictorial representation of a given reoccurence relation, which shows how recurrance is divided till boundary condition (Base condition).

Recursion tree method is specially used to solve a reccurance of the form

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n) \quad \text{—— (1)}$$

where, $a > 1$, $b \geq 1$

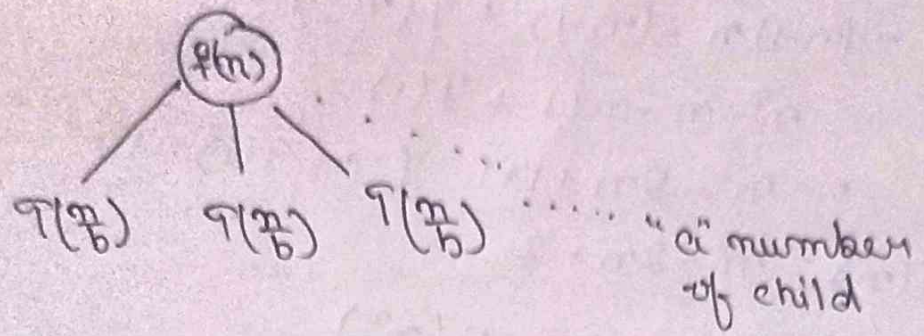
These reoccurance describe the running time of any divide and conquer algorithm.

Steps for solving a reoccurance :—

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n) \quad \text{—— (1)}$$

Step 1:- We make a recurrence tree of a recurrence.

a) First put the value of $f(n)$ at root node of a tree and make "a" number of child node of this root value $f(n)$.



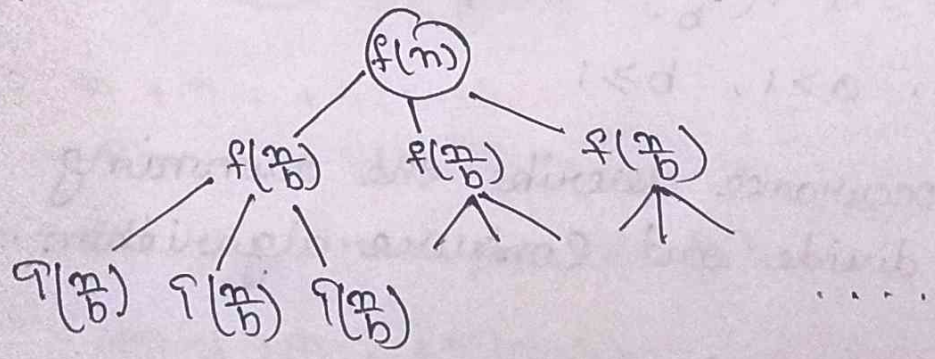
b) Now we have to find the value of $T\left(\frac{n}{b}\right)$ by putting $\left(\frac{n}{b}\right)$ in place of n in eqn -①

$$T\left(\frac{n}{b}\right) = aT\left(\frac{\frac{n}{b}}{b}\right) + f\left(\frac{n}{b}\right)$$

$$T\left(\frac{n}{b}\right) = aT\left(\frac{n}{b^2}\right) + f\left(\frac{n}{b}\right) \quad -②$$

From eqn -②. Now $f\left(\frac{n}{b}\right)$ will be the value of Node having a child node each of size $T\left(\frac{n}{b}\right)$.

Now each $T\left(\frac{n}{b}\right)$ in above figure will be replaced as :-



c) In this way you can expand a tree one more level.

Step 2:-

a) Now you have to find per level cost of a tree. Per level cost is the sum of the cost of each node at that level (i.e Row sum).

b) Now the Total Cost of the tree can be obtain by taking the sum of cost of all these level.

Total Cost = Sum of Cost ($l_0 + l_1 + \cdots$)

c) In this way you can extend a tree up to boundary condition i.e problem size become 1.

Master Method :—

A function $f(n)$ is asymptotically positive if and only if there exist a real no. n such that $f(x) > 0 \ \forall \ x > n$.

The master method provides us state forewarned and cook book methods for solving recurrances of

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$$

where, $a \geq 1$ and $b > 1$ are constant and $f(n)$ is an asymptotically positive function.

This recurrance gives us the running time of algorithm that divides the problem of size n into a sub problem of size $\left(\frac{n}{b}\right)$. The "a" sub problems are solved recursively each in time $T\left(\frac{n}{b}\right)$

The master method require of the following three cases :—

Let $T(n)$ we define on the non-negative integer $T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$ where $a \geq 1$ and $b > 1$, then $T(n)$ can be bounded asymptotically as follows :—

Case 1 :- If $f(n) = O\left(n^{\log_b^a - \varepsilon}\right)$ for some $\varepsilon > 0$ then $T(n) = \Theta\left(n^{\log_b^a}\right)$

Case 2 :- if $f(n) = \Theta\left(n^{\log_b^a}\right)$ Then $T(n) = \Theta\left(n^{\frac{\log_b^a}{\log n}}\right)$

Case 3 :- $\Omega\left(n^{\log_b^a - \varepsilon}\right)$ for some $\varepsilon > 0$ and if $a\,f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ for some constant $c < 1$

and all sufficiently larger, then

$$T(n) = \theta(f(n))$$

nd the complexity of the recurrence.

(b) $T(n) = 9T(\frac{n}{3}) + n$ by using Master method.

$a = 9$
$b = 9$
$f(n) = n$

case 1:

$$\log_b a = \log_3 9 = \log_3(3^2) = 2\log_3 3 = 2$$

$$\therefore \theta(n^{\log_b a}) = (n^2)$$

nd $f(n) = O(n^{\log_b a - \varepsilon})$, where $\varepsilon = 1$

By master theorem case-(1), we get

$$T(n) = \theta(n^{\log_b a})$$

$$T(n) = \theta(n^2)$$

$$T(n) = T(\frac{2n}{3}) + 1$$

$a = 1$
$b = \frac{3}{2}$, $f(n) = 1$

$$\therefore n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$$

$$\therefore f(n) = \theta(n^{\log_b a})$$

By master theorem case-(2) we get

$$T(n) = \theta(n^{\frac{\log_b a}{\log n}})$$

$$T(n) = \theta(n^{\frac{1}{\log n}})$$

Master Method

$T(n) = aT\left(\frac{n}{b}\right) + f(n)$

$a \geq 1, b > 1$

$f(n) = \theta(n^k (\log^p n)^p)$

Case 1: if $\log_b a > k$ then $\theta(n^{\log_b a})$

Case 2: if $\log_b a = k$

if $p > -1$ — $\theta(n^k \log^{p+1} n)$

if $p = -1$ — $\theta(n^k \log\log n)$

if $p < -1$ — $\theta(n^k)$

Case 3: if $\log_b a < k$

if $p \geq 0$ — $\theta(n^k \log^p n)$

$p < 0$ — $O(n^k)$

Q) Find the time complexity of $T(n) = 2T\left(\frac{n}{2}\right) + 1$

Sol

$a = 2$

$b = 2$

$\because 1 = n^0$; Comparing Case - 1

$\therefore k = 0$

$p = 0$

$\therefore \log_2 2 = 1 > k$

$\therefore \theta(n^{\log_b a}) = \theta(n^{\log_2 2}) = \theta(n)$

Q1 $T(n) = 4T\left(\frac{n}{2}\right) + n$

$a = 4$
$b = 2$
$n' = n^k$
$\therefore k = 1$
$p = 0$

$\therefore \log_b a = \log_2 4 = 2$
$\log_b a > k$
$2 > 1$

$\therefore \Theta(n^{\log_b a})$

$\Theta(n^2)$

Q2 $T(n) = 8T\left(\frac{n}{2}\right) + n$

$a = 8, \quad b = 2$

$\therefore k = 3$

Q3 $T(n) = 8T\left(\frac{n}{2}\right) + n^2$

$a = 8, \quad b = 2$
$k = 2 \; ; \; n^2 = n^k$

$\therefore \log_b a = \log_2 8 = 3 > k$

$\therefore \Theta(n^3)$

Q1. $T(n) = 9T\left(\frac{n}{3}\right) + 1$

$a = 9, \quad b = 3$

$n^0 = n^k \Rightarrow k = 1$

Q2. $T(n) = 2T\left(\frac{n}{2}\right) + n$

$a = 2, \quad b = 2$

$\log_b^a = \log_2^2 = 1$

$k = 1$

$\therefore p = 0$

$\therefore \theta(n^k \log^{p+1} n)$

$\theta(n \log^{0+1} n)$

$= \theta(n \log n)$

Q3. $T(n) = 4T\left(\frac{n}{2}\right) + n^2$

$k = 2 ; \quad n^2 = n^k$

$\log_b^a = 2$

$p = 0$

$\theta(n^2 \log n)$

Q4. $T(n) = 4T\left(\frac{n}{2}\right) + n^2 \log n$

$k = 2$

$\log_b^a = 2$

$p = 1$

$\theta(n^2 (\log n)^2)$

Q5. $T(n) = 4T\left(\frac{n}{2}\right) + n^2 \log^2 n$

$k = 2 \quad \log_b^a = 2$

$p = 2$

$\therefore \theta(n^2 \log^3 n)$

Q) $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$

$\log_b^a = 1$

$K = 1$

$\log^{-1} n ; \quad P = -1$

$\therefore \Theta(n^K \log\log n)$

$\therefore \Theta(n \log\log n)$

Q) $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log^2 n}$

$\log_b^a = 1$

$K = 1$

$\log^{-2} n : P = -2$

$\Theta(n^K)$

$\Theta(n)$

Q) $T(n) = T\left(\frac{n}{2}\right) + n^2$

$\log_b^a = 0$

$K = 2 \quad . \quad P = 0$

Case-3 $\therefore$

$\Theta(n^2 \log^0 n)$

$\Theta(n^2)$

Q) $T(n) = 2T\left(\frac{n}{2}\right) + n^2$

$\log_b^a = 1 \quad \log_b^a < 2$

$K = 2$

$P = 0$

$\Theta(n^2)$

Q) $T(n) = 2T\left(\frac{n}{2}\right) + n^2 \log^2 n$

$\log_b^a = 1 < K = 2$

$P = 2$

$\therefore \Theta(n^2 \log^2 n)$

Q) $T(n) = 4T\left(\frac{n}{2}\right) + n^3$

$\log_b^a = 2 \; , K = 3$

$P = 0$

$\Theta(n^3)$

$T(n) = 4T\left(\frac{n}{2}\right) + \frac{n^3}{\log n}$

$\log_b^a = 2 < 3$

$p = -1$

$\Theta(n^3)$

a) $T(n) = 2T\left(\frac{n}{2}\right) + 1$

$\theta(n)$ Ans

b) $T(n) = 4T\left(\frac{n}{2}\right) + 1$

c) $T(n) = 4T\left(\frac{n}{2}\right) + n$

$\theta(n^2)$ Ans

d) $T(n) = 8T\left(\frac{n}{2}\right) + n^2$

$\theta(n^3)$ Ans

e) $T(n) = 16T\left(\frac{n}{2}\right) + n^2$

$\theta(n^4)$ Ans

f) $T(n) = T\left(\frac{n}{2}\right) + n$

$\theta(n)$

g) $T(n) = 2T\left(\frac{n}{2}\right) + n^2 \log n$

$\theta(n^2 \log n)$ Ans

h) $T(n) = 4T\left(\frac{n}{2}\right) + n^3 \log^2 n$

$\theta(n^3 \log^2 n)$

i) $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n^2}{\log n}$

$O(n^2)$

Q) $T(n) = T(\frac{n}{2}) + 1$

$\Theta(n)$

Q) $T(n) = 2T(\frac{n}{2}) + n$

$\Theta(n \log n)$

Q) $T(n) = 2T(\frac{n}{2}) + n \log n$

$\Theta(n \log^2 n)$

Q) $T(n) = 4T(\frac{n}{2}) + n^2$

$\Theta(n^2 \log n)$

Q) $T(n) = 4T(\frac{n}{2}) + (n \log n)^2$

$\Theta(n^2 \log^3 n)$

Q) $T(n) = 2T(\frac{n}{2}) + n \log^{-1} n$

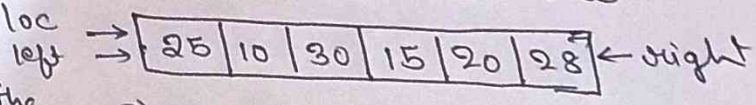
$\Theta(n \log \log n)$ Ans

Q) $T(n) = 2T(\frac{n}{2}) + n \log^{-2} n$

$\Theta(n)$ Ans

Quick Sort :-

Quick Sort is a sorting algorithm use the idea of divide and conquere. This algorithm finds the element called pivot, that partitions the array into two half in such a way that the elements in the left sub array are less than and the elements in the right sub " " greater than the partitong element. Then the 2 ~~sorted~~ sub array sorted separately. This procedure is recursive in nature in the best condition the no. of array are not more than one.

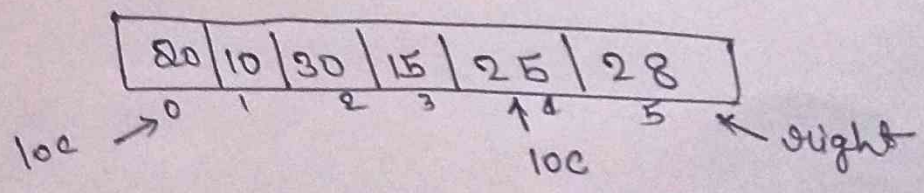


Here the pivot element 25 to with the loc = 0, left = 0 and right = 5

Starting scanning elements from ~~on~~ right. ~~the~~ ~~decreare~~ a[loc] < a[right] we decreare the value of variable right.

a[loc] > a[right]

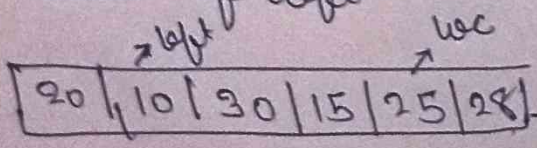
Now, a[loc] is greater a[right] interchange. there element to get



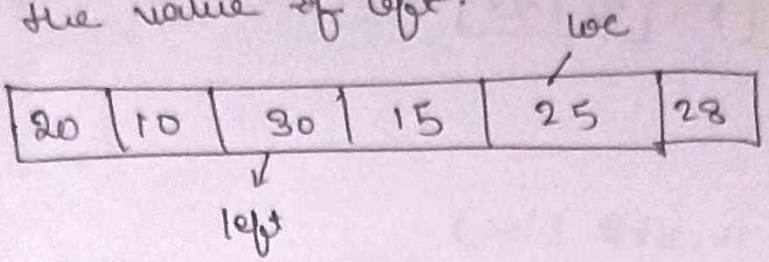
Now starts scanning element from left.

Since a[loc] > a[left]

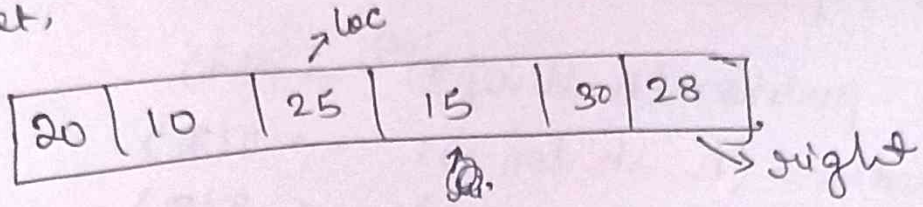
Increase the value of left



Since again a[loc] > a[left] ; Again increase the value of left.

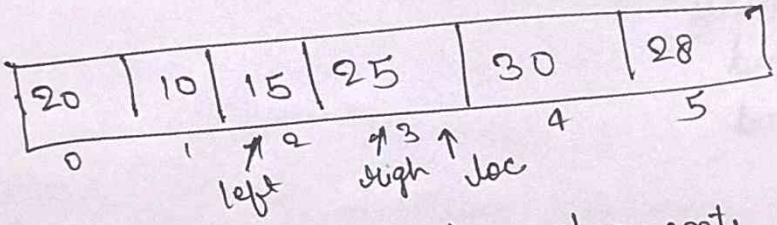


Now a[loc] < a[left], interchange there elements to get,

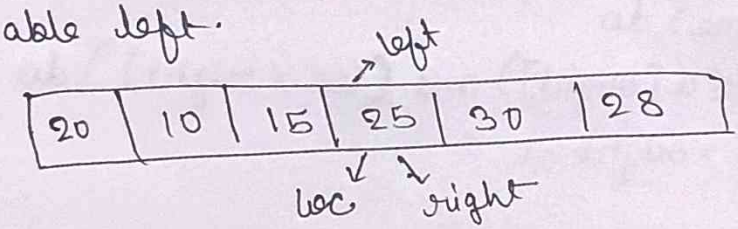


Again start scanning elements from right.
Since a[loc] < a[right], decrease the value of right

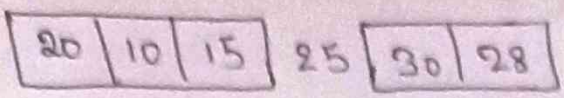
a[loc] > a[right], inter change to get



Now again, start scanning elements from left a[loc] is greater tha a[left] increase the value of variable left.



Since, left become equal to loc, thus indicating the termination of the procedure. Hence the pivot element 25 is placed in the final position and it divides the array into two subarray.




```
QuickSort (A, lb, ub, &loc)
{
  if (lb < up)
  {
    loc = partition (A, lb, ub, loc);  → O(n)
    QuickSort (A, lb, loc-1);          → T(n/2)
    QuickSort (A, loc+1, ub);          → T(n/2)
  }
}
```

Algorithm of Partitioning :-

```
Partition (A, beg, end, loc)
  A - Linear Array
  beg - lower bond
  end - upper bond
Begin
  set left = beg, right = end, loc = beg.
  Set done = false
  while (not done) do
   while (a[loc] ≤ a[right]) and (loc ≠ right)) do
      Set right = right - 1
  end while
  if (loc = right)
  then
      set done = true;
  else if (a[loc] > a[right])
  then
```

interchanging a[loc] & a[right]

Set loc = right

end if

if (not done)

then

Worst Case :-

```
Quick_Sort (A, lb, up, &loc)
{
  if (lb < up)
  {
    loc = partition (A, lb, up, loc);  ——— O(n-1)
    QuickSort (A, lb, loc-1);  ——— T(n-1)
    QuickSort (A, loc+1, ub);  ——— T(0)
  }
}
```

$T(n) = T(n-1) + cn$

$= T(n-1) + O(n)$

$T(n) = T(n-1) + c(n)$; c = constant ——— (1)

Put $n = n-1$

$T(n-1) = T(n-2) + c\,n-1$ ——— (2)

Again, $n = n-1$

$T(n-2) = T(n-3) + c\,n-2$ ——— (3)

Again $T(n-3) = T(n-4) + c\,n-3$ ——— (4)

By Back Substitution Method :-

Binary Search :- (a, i, j, x

a - Linear Array
i - lower bound
j = upper "
x = element to search

```
Begin
  if (i == j)                          ------ O(1)
   if (a[i] == x)
      return i;
   else
      return -1;
  else
   {
     mid = (i+j)/2 ------ O(1)
      if [a[mid] == x) ------ O(1)
        return mid;
      else if (a[mid] > x)------ O(1)
      return BinarySearch(a, i, mid-1, x); ---- T(n/2)
       else
        return BinarySearch(a, mid+1, j, x);
      }
   }
```

$T(n) = T\left(\frac{n}{2}\right) + c$

$= O(\log_2 n)$ — Worst Case

$= O(1)$ ⟶ Best Case

Finding Max^m, Min^m in the given array:

By using divide and conquer strategy:-

```
DACMM(a,i,j, max, min)
{
  a = linear array
  i = lower bond
  j = upper  ''
  max = max^m value
  min = min^m value
}
 if(i==j)
   return max = min = a[i];
 else if(i==j-1)
 {
   if(a[i] < a[j])
   {
     max = a[j];
     min = a[i];
   }
   else{
     max = a[i];
     min = a[j];
   }
 }
else
 {
  mid = (i+j)/2;  ——— 2
 DACMM(a, i, mid, max, min); — T(n/2)
 DACMM (a, mid+1, j, max1, min1); — T(n/2)
  if(max1 > max)
    max = max1;
  if(min1 < min)
    min = min1;
 }
}
```

Now analysing:-

$T(n) = 0$ if $n = 1$

$= 1$ if $n = 2$

$T(n) = 2T(\frac{n}{2}) + C$ if $n > 2$

Time Complexity $= O(n)$

Greedy Method

A technique to solve optimization problem.

Greedy method are typically used to solve an optimization problem.

An optimization problem is one in which we are given a set of input values, which are required to be either maximized or minimize with respect to some constraints or some conditions.

Generally an optimization problem has n inputs (~~n no. of inputs~~). Call this set an input domain or Candidate set (C). We are required to obtain a subset of C (called a solution set), S where S is subset of C. (that satisfy the given constraints for condition). Any subset S subset of C which satisfy the given constraint is called Feasible solution.

We need to find a feasible solution that maximizes or minimizes a given objective function. The feasible solution that does, this is called an Optimal solution.

Algorithm of Greedy Method :-

General form of Greedy Algorithm.

```
Greedy (c,n)
/* input : A input domain(or Candidate set) C of
           size n */
// function select (C: Candidate set) return an element
   ( or Condidate).
// solution (S: Condidate set) return Boolean
// function feasible (S: Candidate set) return Boolean
/* Output — A solution set S where S ⊆ C which
           maximize or minimize the selection
           Criteria w.r.t given constraints */
{
   S ← ϕ // initially a solution set S is empty.
   while( not solution(S) and C ≠ ϕ)
   {
      x ← select (C) // A best element x is selected
                     // from C(Candidate set) which
                     // maximize or minimize the selection
                     // criteria.
      C ← C - {x} /* Once x is selected it is removed
                     from C */
      if( feasible(S U {x})) then
      {
         S ← S U {x}
      }
      if( solution (S))
         return S;
      else
         return "No solution"
   }
```

Q. Suppose we are given Indian currency notes of all denomination (1, 2, 5, 10, 20, 50, 100, 200, 500, ~~2000~~), the problem is to find the minimum number of currency notes to makes the required amount 389 for payment. Q. It is assumed that currency notes of each denomination are available in sufficient number. So that one may choose as many notes of the same denomination as are required for the purpose of min^m no. of notes to make the amount 389.

Sol
We pick up a note of denomination of D satisfying the condition if D ≤ 389
and if D1 is another denomination of note such that D1 ≤ 389 then D1 ≤ D

Choose Denomination	Total value
500 ✗	0+500 > 389 ✗
✓200	1 + 200 < 389
200 ✗	200+200 > 389
✓100	200+100 < 389
100 ✗	100+200+100 > 389
✓50	50+200+100 < 389
✓20	20+50+200+100 < 389
✓10	10+20+50+200+100 < 389
✓5	
2	
2	
1	

KnapSack Problem

The fractional knapSack problem is defined as :-

1. Given a list of n objects say $(I_1, I_2, I_3, \cdots I_n)$ and a knapSack (bag).
2. Capacity of knapSack is M.
3. Each object has a weight (w_i) and profit (P_i).
4. If a fraction x_i (where $x_i \in (0, \cdots 1)$ of an object I_i is placed into a knapSack then the profit of $P_i x_i$ is earned.

The objective is to fill the knapSack (upto its max^m capacity n) which maximizes the total profit earned.

Mathematically :—

Maximize (profit) $\sum_{i=1}^{n} P_i x_i$ subjected to constraints

$\sum_{i=1}^{n} w_i x_i \leq M$ and $x_i \in (0, \cdots 1)$ $1 < i \leq n$

Example :—

Objects (O) :	1	2	3	4	5	6	7
Profit (P) :	10	5	15	7	6	18	3
weight (w) :	2	3	5	7	1	4	1

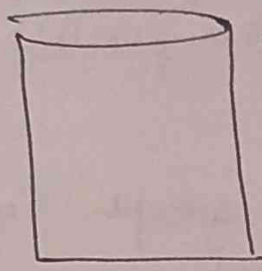
Capacity of knapSack (M) = 15

Sol

$\frac{P}{w}$ = 5 1.6 3 1 6 4.5 3

x(x_1 x_2 x_3 x_4 x_5 x_6 x_7)

(2/3)



15 - 1 = 14
14 - 2 = 12
12 - 4 = 8
8 - 5 = 3
3 - 1 = 2
2 - 2 = 0

$$\sum_{i=1}^{n} w_i x_i = (1)(2) + \left(\frac{2}{3}\right)(3) + (1)(5) + 0 + (1)(1) + (1)4 + (1)(1)$$

$$= 2 + 2 + 5 + 1 + 4 + 1$$

$$= 15 \leq M \quad \text{satisfied}$$

(b)

$$\sum_{i=1}^{n} p_i x_i = (10)(1) + (5)\left(\frac{2}{3}\right) + 15(1) + 7(0) + 6(1) + 18(1) + 3(1)$$

$$= 10 + \frac{10}{3} + 15 + 6 + 18 + 3$$

$$= 55.3333 \text{ which is Maximum.}$$

Graph and Tree :-

Spanning Tree :-

Let G = (V, E) be an undirected connected graph. A sub graph T = {V, E'} of G is a spanning tree of G if and only if T is a tree (No cycle exist) and contains all the vertices of G.

Minimum Cost Spanning Tree (MCST) :-

A MCST of a weighted connected graph G is that spanning tree where sum of length (weight) of all its edges is minimum, among all the possible spanning tree of G.

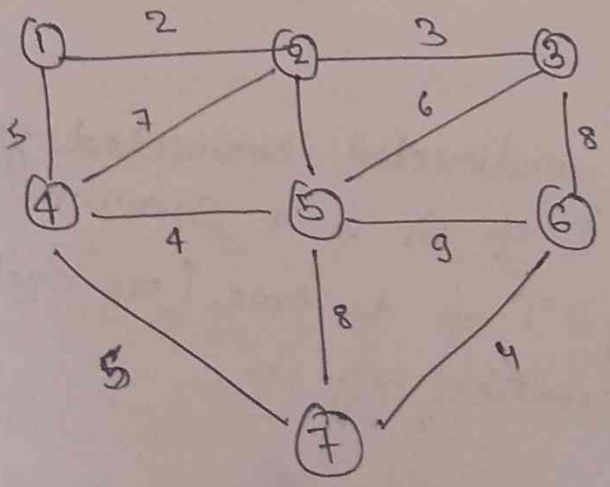
To find MCST ~~we have~~ of a given graph G, one of the following algorithm is used.

(1) Kruskal's Algorithm
(2) Prim's Algorithm

There two algorithm use greedy approach. A greedy algorithm selects the edges one by one in some given order. The next edge to include is chosen according to some optimization criteria.

(1) Kruskal's Algorithm :-

Apply Kruskal's Algorithm on the following graph to find min^m cost spaning tree.



We sort the edges of $G_1 = (V, E)$ in order of increasing weights as

Edges	(1,2)	(2,3)	(4,5)	(6,7)	(1,4)	(2,5)	(4,7)	(3,5)	(2,4)	(3,6)
Vertices	2	3	4	4	5	5	5	6	7	8

Step	Edge Considered	Connected Component	Spanning forest (s)
Initialization	—	{1}{2}{3} ··· {7}	(1)(2)(3) ··· (7)
1	(1,2)	{1,2}{3}{4} ··· {7}	(1)-(2) (3) ··· (7)
2	(2,3)	{1,2,3}{4} ··· {7}	(1)-(2)-(3) (4) ··· (7)
3	(4,5)	{1,2,3}{4,5}{6}{7}	(1)-(2)-(3) (6) (7) (4)-(5)
4	(6,7)	{1,2,3}{4,5}{6,7}	(1)-(2)-(3) (6) (7) (4)-(5) (6) (7)
5	(1,4)	{1,2,3}{4,5}{6,7}	(1)-(2)-(3) (4)-(5) (6) (7)
6	(2,5)	Edge (2,5) is rejected because it make cycle	(1)-(2)-(3) (4)-(5) (6) (7)
7	(4,7)	{1,2,3,4,5,6,7}	(1)-(2)-(3) (4)-(5) (6) (7)

Let G(V,E) is a connected weighted graph, in Kruskal algorithm, first we examine the edges of G in order of increasing weight. Then we select an edge $(u,v) \in E$ of min^m weight and check whether its ends point belongs to same component or not different connected component.

If u and v belongs to different connected components then we add it to set it, otherwise it is rejected because it creates a cycle.

The algorithm stops when only one connected component remains.

Pseudo-Code of Kruskal's Algorithm

```
KRUSKAL_MCST(G, w)
// Input: A undirected Connected weighted Graph G=(V,E)
// Output: A minm Cost Spanning Tree T(V,E') of G
{
1. Sort the edges of E in order of increasing weight
2. A ← ϕ
3. For (each vertex V ∈ V[G])
4. do MAKE_SET(v)
5. For (each edge (u,v) ∈ E taken in increasing order
     of weight.
   {
6. if (FIND_SET(u) ≠ FIND_SET(v))
7.    A ← A ∪ {(u,v)}
8.  MERGE (u,v)
   }
9. return A
}
```

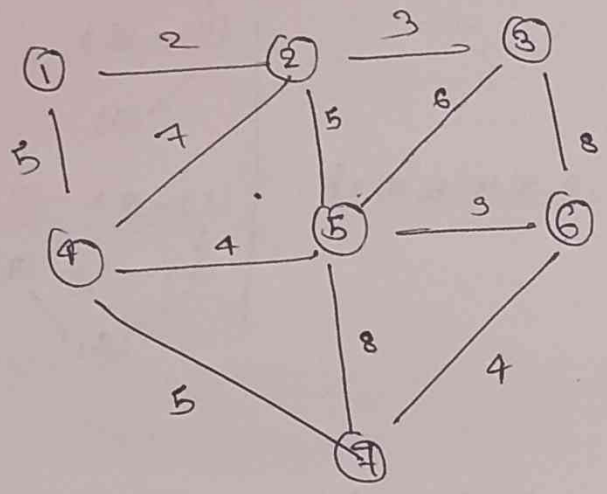
Prim's Algorithm :-

* Making with one starting vertex (say v) of the given graph G=(V,E).

* Then in each iteration we chose a minimum weightage (u,v) connecting a vertex v in the shape A to the vertices in the set (V, -A) i.e we always find an edge (u,v) of minimum weight such that $v \in a$ and $u \in v-A$, then we modify the set a by adding u.

$$A \leftarrow A \cup \{u\}$$

* This process is repeated until all the vertices are not in the ~~same~~ set A.



Edge	
Vertices	

Step	Edge Connected	Connected Component	Spanning Forest (T)
Initialize	____	{1}	(1)
1.	(1,2)	{1,2}	(1)-(2)
2.	(2,3)	{1,2,3}	(1)-(2)-(3)
3.	(1,4)	{1,2,3,4}	(1)-(2)-(3) | (4)
4.	(4,5)	{1,2,3,4,5}	(1)-(2)-(3) | (4)—(5)
5.	(~~4~~,7)	{1,2,3,4,5,~~7~~}	(1)-(2)-(3) | (4)—(5) \ (6)
6	(6,7)	{1,2,3,4,5,6,7}	(1)-(2)-(3) | (4)—(5) (7) \ / (6)

∴ Total Cost = (2 + 3 + 5 + 4 + 5 + 4)

= 23.

Pseudo-Code of Prim's Algorithm

PRIMS_MCST (G, w)

//

//

{

T ← ϕ

A ← {Any arbitary member of V}

3. while ($A \neq V$)

4. {
find an edge (u,v) of $\min^m$ weight $s.t. u \in V-A$ and $v \in A$

$A \leftarrow A \cup \{u,v\}$

$B \leftarrow B \cup \{u\}$

}

5. return T

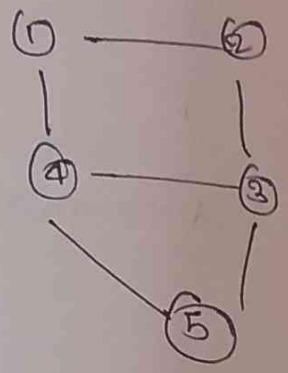
}

Difference b/w Kruskal and Prim's Algorithm

Algorithm on Graph

A graph is non-linear data structure consisting of nodes and edges. Nodes are also refers to as vertices and the edges are lines or arcs that connected any two nodes in a graph

$G = (V, E)$



$V(G) =$

Graph can be represented by following two methods:—

1. Adjacency Matrix
2. " List

1. **Adjacency Matrix:—** Adjacency matrix is a 2D array of $V \times V$ where V = no. of vertices in a graph. Adjacency of $adj[i][j] = 1$ indicates that there is an edge from vertex i to vertex j.

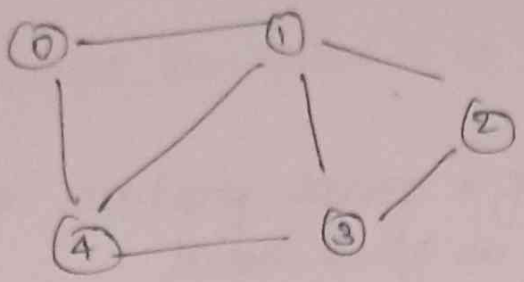
Adjacency matrix for undirected graph is always symmetrix.

Adjacency matrix is also used to represent weighted

graph

adj [i][j] = w

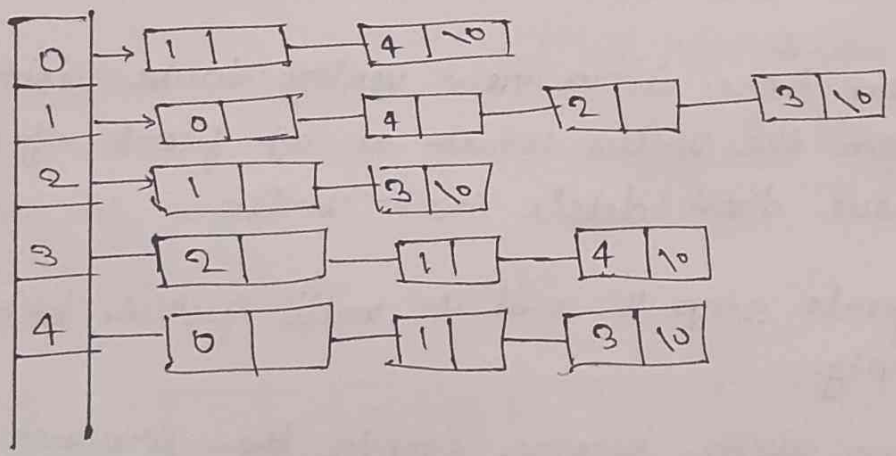
where, w = weight of edge from vertex i to vertex j



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4					

Adjacency List :—

An array of list is used with side should be equals to the number of vertices. An entry A[i] represents the list of vertices adjacent to the i^{th} vertice.



Traversal of Graph :-

There are two Graph Traversal technique

(1) BFS : Best first
(2) DFS

(1) BFS :- BFS traversal of a graph produces a spanning tree as final result.

We use queue data structure with max^m size of total no: of vertices in the graph to implement BFS traversal.

We use the following steps to implement BFS traversal :—

Step 1 :- Define a queue of size total no. of vertices of graph.

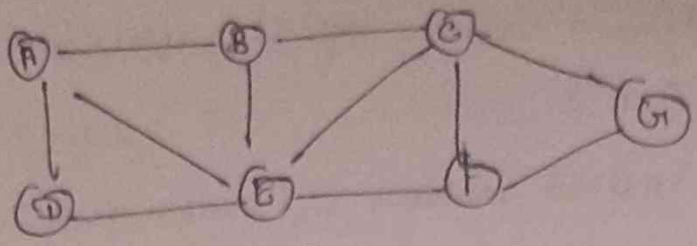
Step 2 :- Select any vertex as starting point for traversal. Visit that vertex and insert it into the queue.

Step 3 :- Visit all the non-visited adjacent vertices of the vertex which is at front of the queue. and insert them into the queue.

Step 4 :- When there is no new vertex to be visited from the vertex which is at front of the queue then delete that vertex.

Step 5 :- Repeate step 3 and 4 until queue becomes empty.

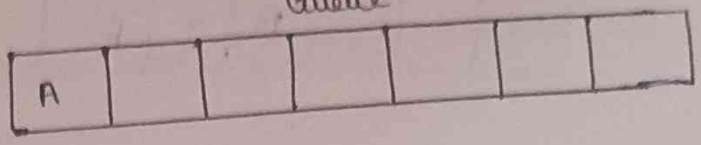
Step 6 :- When queue become empty then produce final spanning tree by removing unused edges in the graph.

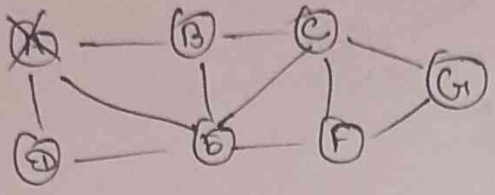


Step 1 :- Select the vertex A as starting vertex (visit A)

↳ Insert A

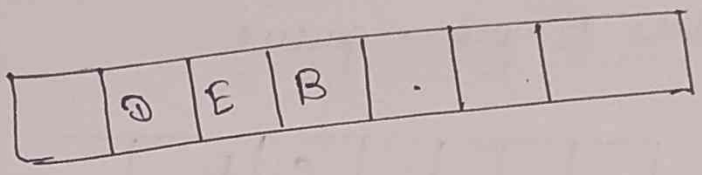
Queue

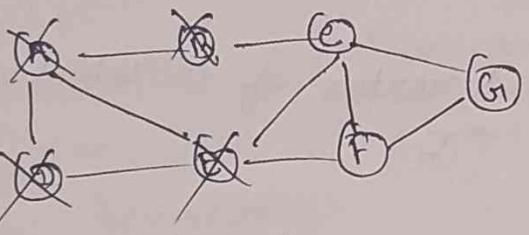




Step 2 :- Visit all all adjacent vertices of A which are not visited. (B, D, E)

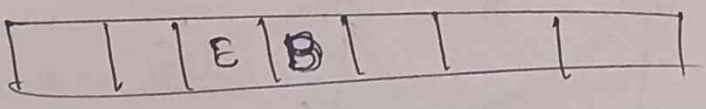
Insert every visited vertex into Queue and delete A





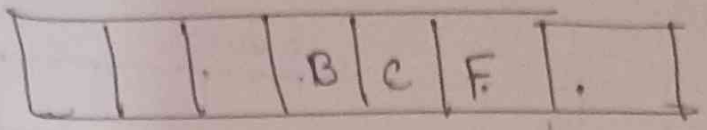
Step 3 :- Visit all adjacent vertex of D which are not visited. (There is no vertex to be visited, because all adjacent vertex of D are visited).

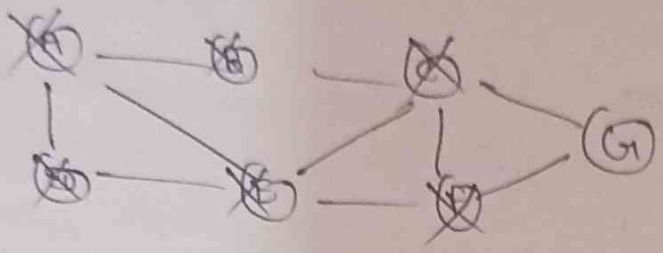
↳ Delete D from queue.



Step 4:- Visit all adjacent vertices of B which are not visited (C, F)

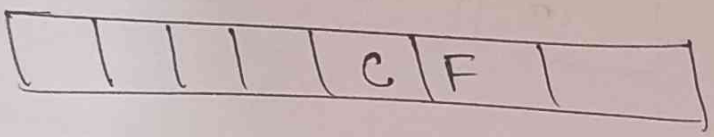
↳ Insert newly visited vertices C, F into queue and delete B





Step 5:- Visit all adjacent vertex of B which are not visited (There is not vertex)

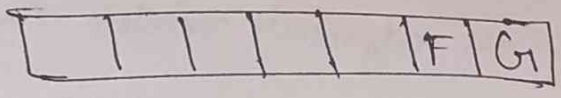
↳ Delete B from queue

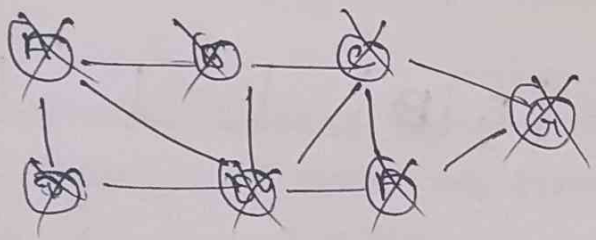


Step 6:- Visit all adjacent vertex of C which are not visited (G1)

↳ Insert newly visited vertex G1 into queue.

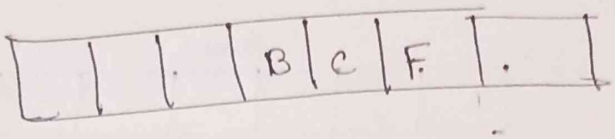
↳ Delete vertex C

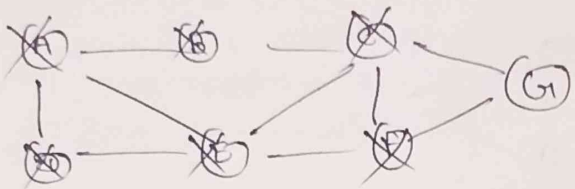




Step 4:- Visit all adjacent vertices of B which are not visited (C, F)

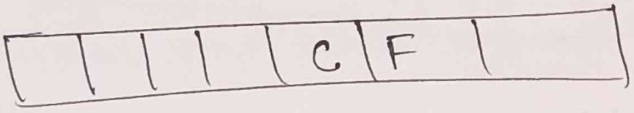
↳ Insert newly visited vertices C, F into queue and delete B





Step 5:- Visit all adjacent vertex of B which are not visited (There is not vertex)

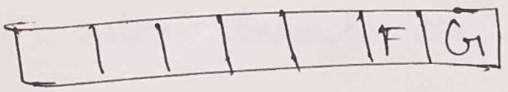
↳ Delete B from queue

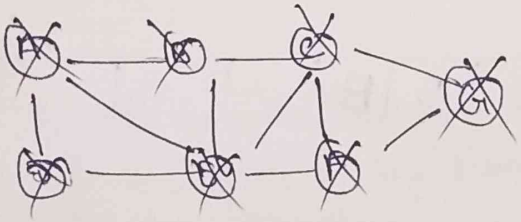


Step 6:- Visit all adjacent vertex of C which are not visited (G1)

↳ Insert newly visited vertex G1 into queue.

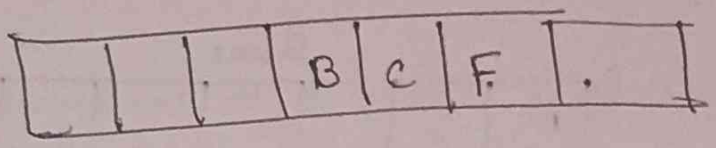
↳ Delete vertex C

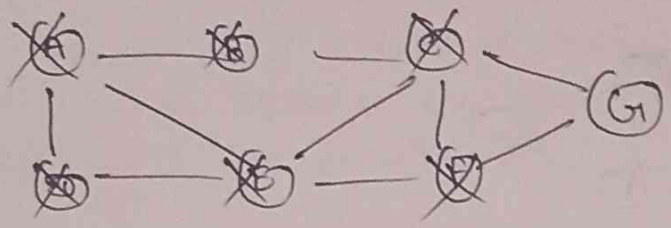




Step 4:- Visit all adjacent vertices of B which are not visited (C, F)

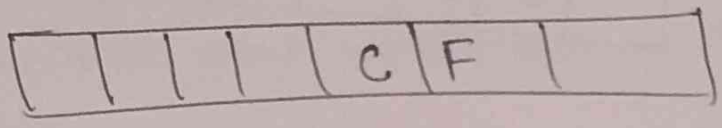
↳ Insert newly visited vertices C, F into queue and delete B





Step 5:- Visit all adjacent vertex of B which are not visited (There is not vertex)

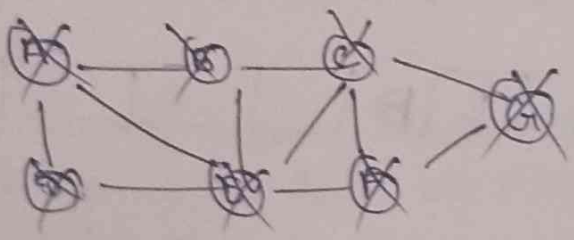
↳ Delete B from queue



Step 6:- Visit all adjacent vertex of C which are not visited (G1)

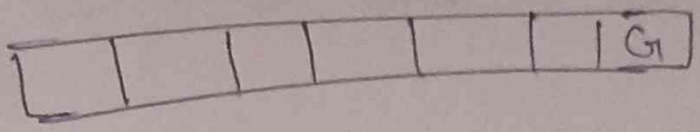
↳ Insert newly visited vertex G1 into queue.

↳ Delete vertex C



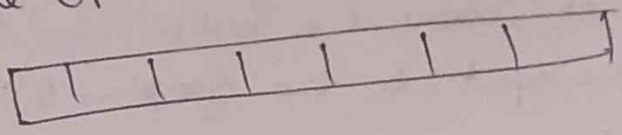
Step 7 :- Visit all ~~vis~~ non-visited vertex of F (No vertex)

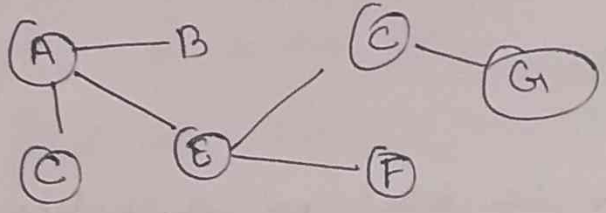
↳ Delete F



Step 8:- Visit all non-visited vertex of G1
(No vertex

↳ Delete G1.





Algorithm :-

BFS (vertices, start)
// input : The list of vertices and the start vertex
// output : Traverse all of the nodes, if the graph is connected.

Begin
Define an empty queue QUE
at first mark all vertex starts a unvisited.
Add the start vertex into the QUE
while QUE is not empty do
delete item from QUE and set to ue
display the vertex u
for all vertices adjacent with u, do

if vertices (i) is unvisited

then

mark Vertex(i) as temporary visited

add v into the queue

mark

done

mark u as completely visited.

BFS has a running time O

will be checked once, where, B is vertices and E is edges.

Depending on the input to the graph, O(E) could be b/w O(1) and $O(B^2)$.

(2) DFS (Depth First Search) :-

Step 1:- Define a stack of size total no. of vertices in the graph.

Step 2:- Select any vertex as starting point for traversal. Visit that vertex and push it on to the stack.

Step 3:- Visit any any one of the non-visited adjacent vertices of ~~the~~ a vertex at which is at top of stack and PUSH it on to the stack.

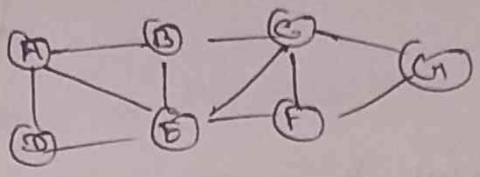
Step 4:- Repeate step 3 untill there is no new vertex to be visited from the vertex which is at the top of the stack

Step 5:- When there is no new vertex to visit then use back tracking and pop one vertex from

Step 6:- Repeat step 3, 4 & 5 until stack become empty.

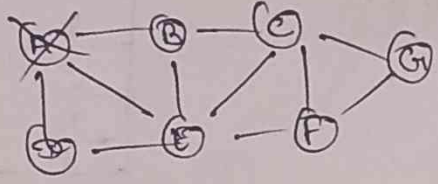
Step 7:- When stack becomes empty, then produce final spanning tree by removing unused edges from the graph.

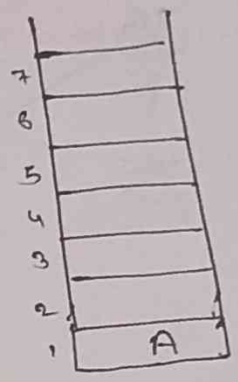
* Back Tracking is coming back to the vertex from which we reach the current vertex.



Step 1:- Select the vertex (A) as starting point (visit A)

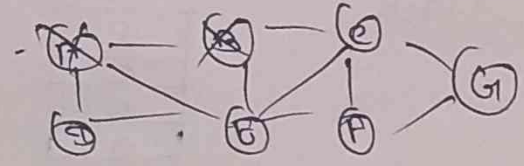
↳ PUSH it A on to the Top

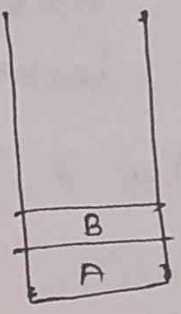




Step 2:- Visit any adjacent vertex of A which is not visited (visit B)

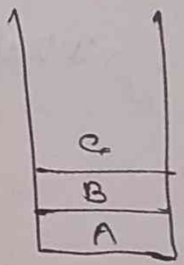
PUSH B on to the stack



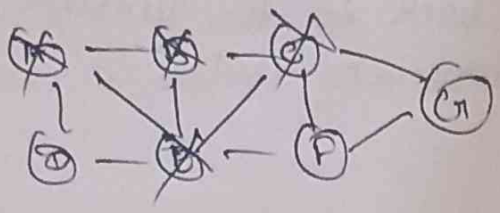


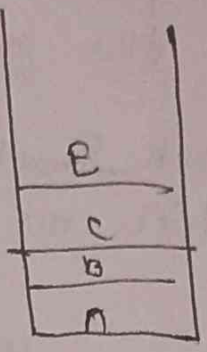
Step 3:- Visit any adjacent vertex of B which is not visited (visit C).

↳ PUSH C on to the stack

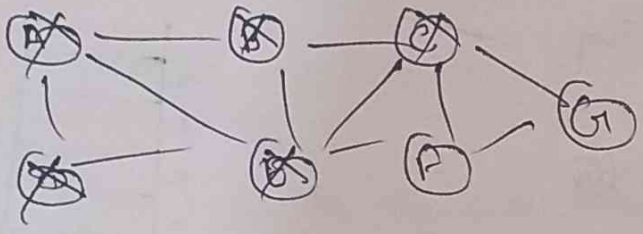


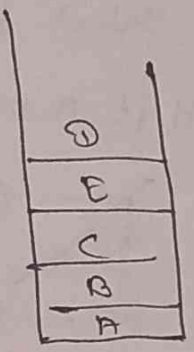
Step 4 :- Visit any adjacent vertex of C which is not visited (visited E)





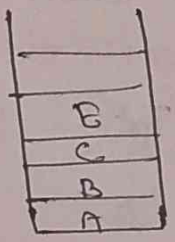
Step 5 :- Visit any adjacent vertex of E which is not visited (visit D)



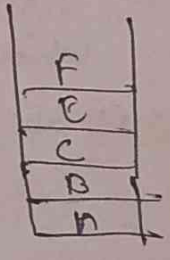


Step 6 :- There is no new vertex to be visited from D, so we backtracking

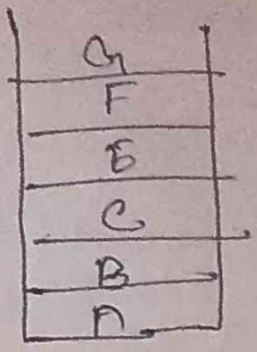
↳ Pop D from the stack

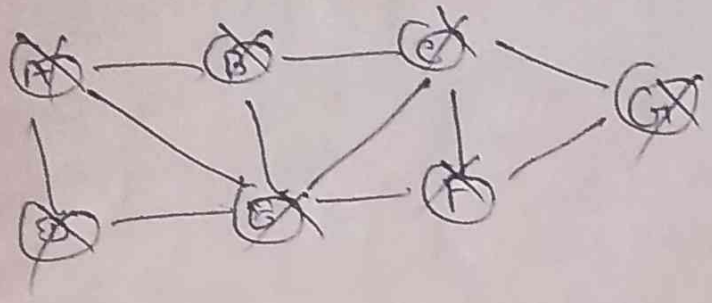


Step 7 :- Visit any adjacent vertex of B which is not visited (visit F)



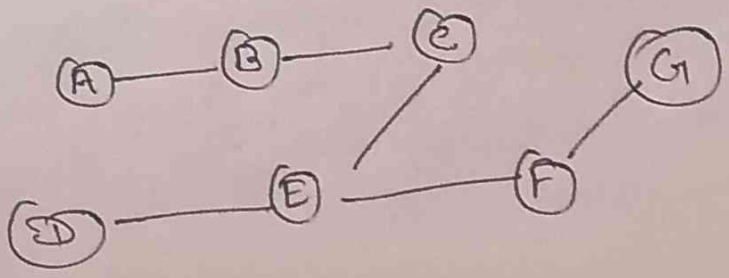
Step 8 :- Visit any adjacent vertex of F which is not visited (visit G)





Step 9 :- There is no adjacent vertex of G so we back tracking

↳ Pop G from stack
↳ Pop F, E, C, B, A



DFS Algorithm :-

Hashing is useful searching technique designed using mathematical model of function. It realies on the concept of Hash table.

Hash Table :- is a data structure in which the location of data items is determined directly as a function of the data ~~item~~ item itself. Data item in hash table are stored in such a way as to make it easy to find them. Each position of the hash table, often called a slot, can hold an item and is named by an integer value starting at zero.

Hash function :- A Hash function h is simply a mathematical formula that manipulates the key in some form to compute the index of this key in the hash table.

Type of Hash function :-

The main Hash functions are :-

1. Division Method
2. Multiplication "
3. Mid-a Square Method
4. Folding Method.

(1) In Division Method Key 'K' to be mape into one of the n slots in the Hash Table is divided by m and the reminder of this division is taken as index in to the Hash Table.

③ Multiplication Method :— It operates in two steps

③ Mid Square Method operates in two steps in the first step the square of the key value k is taken. In the second step the Hash value is obtained by deleting digits from i.e. k^2.

It is important to note that same position of k^2 must be used for all key

$$h(k) = s$$

where, s is obtained by deleting digits from both sides of k^2.

Ex:- Consider a Hash Table with 100 slots i.e $m = 100$ and key values $k = 3205, 7148, 2345$

k	= 3205	7148	2345
k^2	= 10272025	51093904	5499025
h(k)	= 72	93	90

Here, Hash value are obtained by taking 4th and 5th digit counting from ~~side~~ right.

④ The Folding method, also operates in two steps.

Step 1:- The key value k divided into number of parts $k_1, k_2, k_3 \ldots k_r$, where each part has the same number of digit except the last part which can have lesser digit.

Step 2:- In the 2nd step these parts are added together and a hash value is obtained by removing the last carry if ~~any~~ else.

For ex:- If the Hash table has 1000 slots e part have three digit and the sum of these parts after ignoring the last is also a 3 digit in the range 0 to

Q: Consider a hash table with 100 slots and ke value 9235, 714 and 71458

K =	9235	714	71458
parts =	92, 35	71, 4	71, 45, 8
Sum =	92+35 = 127	71+4 = 75	71+45+8 = 124
h(K)	27	75	24

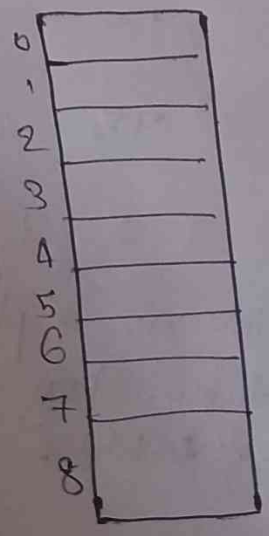
Collision:- A collision is a phenomenon that occurs when more than key maps to the same

Collision dissolution Method :—

(I) Chaining
(II) Linear Probing
(III) Quadratic "
(IV) Double hashing

(I) In this scheme all the elements whose keys hash to the same Hash Table slot are put in a one link list. Thus the slot i in the Hash Table contains of pointer to the head of the link list of all the elements that hashes to the value i, if there is no such element that hash value ~~the~~ i, the slot i contain NULL value.

Ex:- Let us consider the insertion of elements 5, 28, 19, 15, 20, 33, 12, 17 and 10. into a chain hash table. Let us suppose Hash table has 9 slots and the hash function be $h(K) = K \bmod 9$.



(2) Linear P

Collision resolution by Open addressing

In the open addressing scheme all the elements of the dynamic set are stored in the Hash Table itself. In order to insert a key in the hash table under this scheme, if the slot to which key is hashed is free then the element is stored at that slot. In case the slot is filled then other slots are examine systematically in the forward direction to find the free slots. If no such slot is found then overflow condition occurs.

The process of examine the slot in the Hash Table is called probbing.

(1) Linear Probbing :- The L.P use the following Hash function

$$h(K,i) = (h'(K) + i) \bmod m$$

$i = 0, 1, 2, 3, \ldots, (m-1)$

m = Size of Hash Table

$h'(K) = K \bmod m$

Ex Consider inserting the keys 76, 26, 37, 59, 21, 65, 68 into a Hash table of size $m = 11$. using linear probing.

Sol

$h'(K,i) = [h'(K) + i] \bmod m$

i) 76

$[76 \bmod 11 + 0] \bmod 11 = 10$

ii) 26

$[26 \bmod 11 + 0] \bmod 11 = 4$

iii) 37

$[37 \bmod 11 + 0] \bmod 11 = 4$ (Collision)

$i = 1$

$[37 \bmod 11 + 1] \bmod 11 = 5$

iv) 59

$[59 \bmod 11 + 0] \bmod 11 = 4$

$i = 1$

$[59 \bmod 11 + 1] \bmod 11 = 5$

$i = 2$ $= 6$

v) 21

$[21 \bmod 11 + 0] \bmod 11 = 10$

$i = 1$

$[21 \bmod 11 + 1] \bmod 11 = 0$

vi) 65

$[65 \bmod 11 + 0] \bmod 11 = 10$

$i = 1$

$[65 \bmod 11 + 1] \bmod 11 = 00$

$i = 2$ $= 1$

0	21
1	65
2	88
3	
4	26
5	37
6	59
7	
8	
9	
10	76

i) 76

$[76 \bmod 11 + 0] \bmod 11 = 10$

ii) 26

$[26 \bmod 11 + 0] \bmod 11 = 4$

iii) 37

$[37 \bmod 11 + 0] \bmod 11 = 4$ (Collision)

$i = 1$

$[37 \bmod 11 + 1] \bmod 11 = 5$

iv) 59

$[59 \bmod 11 + 0] \bmod 11 = 4$

$i = 1$

$[59 \bmod 11 + 1] \bmod 11 = 5$

$i = 2$

$= 6$

v) 21

$[21 \bmod 11 + 0] \bmod 11 = 10$

$i = 1$

$[21 \bmod 11 + 1] \bmod 11 = 0$

vi) 65

$[65 \bmod 11 + 0] \bmod 11 = 10$

$i = 1$

$[65 \bmod 11 + 1] \bmod 11 = 0$

$i = 2$

$= 1$

0	21
1	65
2	88
3	
4	26
5	37
6	59
7	
8	
9	
10	76

from the problem called primary clustering. Here by a cluster we mean a block of occupying slots and primary ~~contraction~~ clustering refers to many such block separated by free blocks.

(2) Quadratic Probbing:-

To avoid problem of primary clustring we can use Quadratic probbing and double hashing.

$$h'(k,i) = [h'(k) + c_1 i + c_2 i^2] \bmod m$$

$i = 0, 1, 2, 3, \ldots m-1$

m = Size of hash table

$h'(k) = k \bmod m$

c_1 and $c_2 \neq 0$ are auxilary constant

i = probe no.

Ex:- Insert 76, 26, 37, 59, 21, 65, 88 ; $c_1 = 1, c_2 = 3$

i) 76

$[76 \bmod 11 + 0 + 0] \bmod 11 = 10$

ii) 26

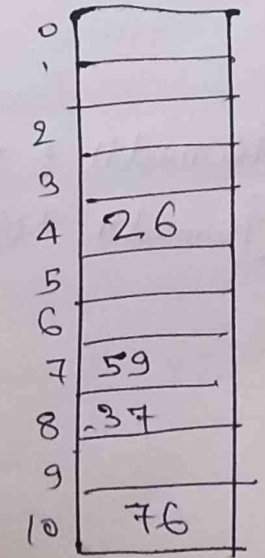
$[26 \bmod 11 + 0 + 0] \bmod 11 = 4$

iii) 37

$[37 \bmod 11 + 0 + 0] \bmod 11 = 4$

$i = 1$

$[37 \bmod 11 + 1 + 3] \bmod 11 = 8$



59

$[59 \bmod 11 + 1(2) + 3(4)] \bmod 11$

$[4 + 2 + 12] \bmod 11 = 7$

Double Hashing :- It is one of the best method for open addressing ~~where~~ because the permutation produce have many of the features of randomly chosen permutation.

Double Hashing use the hash function

$$h(k,i) = [h_1(k) + i \cdot h_2(k)] \bmod m$$

$h_1(k) = [k \bmod m]$

$h_2(k) = [k \bmod m']$

m' = are chosen to be slightly less than m i.e $m-1$ or $m-2$

Ex :-

$[76 \bmod 11 + 0] \bmod 11$

$[76 \bmod 11 + 0(76 \bmod 10)]$

Set:- A set is a collection of distinct elements.

Eg:- $S1 = \{1, 2, 5, 6\}$

$S2 = \{3, 7, 8\}$

$S1 \cap S2 = \phi$

Disjoint Set:- The disjoint set are those who does not have any common element.

Its operations are:—

i) Union

ii) ~~ϕ(fi)~~ find

(i) If S_i and S_j are two disjoint set then their union $S_i \cup S_j$ consists of all the element $x \in S_i$ or $x \in S_j$.

$S1 \cup S2 = \{1, 2, 3, 5, 6, 7, 8\}$.

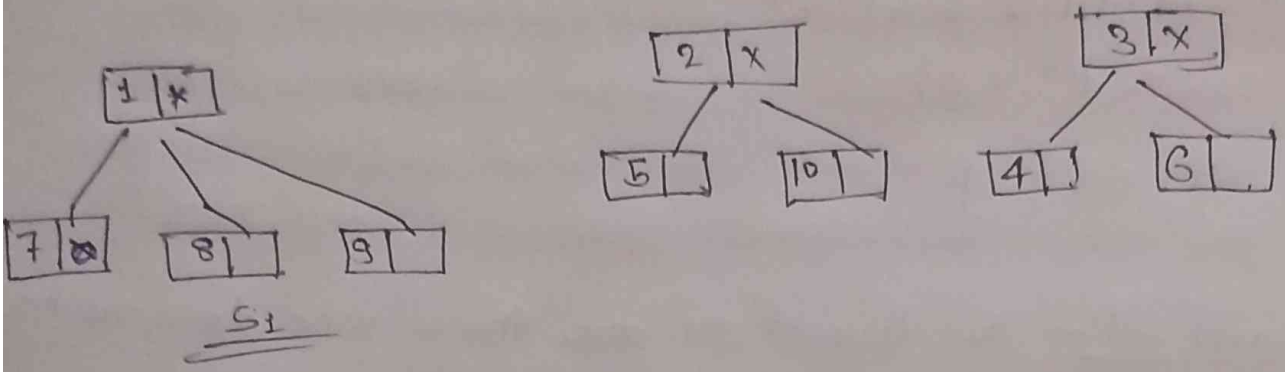
(ii) Find:- Given the element i find the set containing i.

find (5) = S1

find (8) = S2

The set representation :- The set is represented as the ~~tree~~ tree structure where all children will store the address of parent of root node. The root node will store null at the place of parent address. In the given set of element any element can be selected as the root node. generally will select the first node as root node.

$S_1 = \{1, 7, 8, 9\}$ $S_2 = \{2, 5, 10\}$, $S_3 = \{3, 4, 6\}$



S_1

Disjoint Union :- To perform disjoint set union between two set S_i and S_j can take any one root make it subtree of others. Consider the above example S_1 and S_2 then the union of S_1 and S_2 can be represented as any one of the following.